

java.util.Arrays — Method Cheat Sheet

Method	Purpose	Time	Space	Imp.
sort()	Sort array	$O(n \log n)$	Prim: $O(\log n)$, Obj: $O(n)$	*****
binarySearch()	Binary search sorted arr	$O(\log n)$	$O(1)$	*****
fill()	Fill w/ value	$O(n)$	$O(1)$	****
copyOf()	Copy/resize array	$O(n)$	$O(n)$	****
copyOfRange()	Copy [<i>from</i> , <i>to</i>)	$O(k)$	$O(k)$	****
equals()	Compare 1D arrays	$O(n)$	$O(1)$	***
deepEquals()	Compare nested arrays	$O(n)$ total	$O(d)$ recur.	*
toString()	Print 1D array	$O(n)$	$O(n)$	****
deepToString()	Print nested array	$O(n)$ total	$O(n)$	*
asList()	Array → Fixed List	$O(1)$	$O(1)$	****
clone()	Shallow copy	$O(n)$	$O(n)$	**

1. sort()

Primitive

```
int[] arr = {5,2,8,1};
Arrays.sort(arr);
```

```
[1, 2, 5, 8]
```

Objects (Comparator)

```
Arrays.sort(students,
    Comparator.comparingInt(s -> s.marks));
```

2. binarySearch()

```
int idx = Arrays.binarySearch(arr, 5);
```

```
≥ 0 => found    < 0 => not found
```

Range version

```
Arrays.binarySearch(arr, 2, 7, 5);
```

```
Searches range [2,7)
```

3. fill()

Entire array

```
Arrays.fill(arr, 0);
```

```
0 0 0 0
```

Range

```
Arrays.fill(arr, 2, 5, -1);
```

```
Fills range [2,5)
```

4. copyOf()

```
int[] copy = Arrays.copyOf(arr, 7);
```

```
Original: 1 2 3
Result:   1 2 3 0 0 0 0
```

5. copyOfRange()

```
Arrays.copyOfRange(arr, 2, 6);
```

```
Copies range [2,6)
```

6. equals()

```
Arrays.equals(a, b);
```

```
{1,2,3} vs {1,2,3} => true
```

7. deepEquals()

```
int[][] a = {{1,2},{3,4}};
int[][] b = {{1,2},{3,4}};
Arrays.deepEquals(a, b); // true
```

8. toString()

```
System.out.println(Arrays.toString(arr));
```

```
[1, 2, 3]
```

9. deepToString()

```
System.out.println(
    Arrays.deepToString(matrix));
```

```
[[1, 2], [3, 4]]
```

10. asList()

```
Integer[] arr = {1,2,3};
List<Integer> list = Arrays.asList(arr);
```

Remember: Fixed-size list.
list.add(4); => **UnsupportedOperationException**

11. clone()

```
int[] copy = arr.clone();
```

```
New array, same elements → shallow copy.
```

Comparator

Ascending

```
Comparator<Integer> cmp = Integer::compare;
// or
(a, b) -> Integer.compare(a, b)
```

Descending

```
Comparator<Integer> cmp =
    (a, b) -> Integer.compare(b, a);
// or
Comparator.reverseOrder();
```

Student (single key)

```
Comparator<Student> byMarks =
    Comparator.comparingInt(s -> s.marks);
```

Descending on key

```
Comparator<Student> byMarks =
    Comparator.comparingInt(
        (Student s) -> s.marks).reversed();
```

Multiple keys

```
Comparator<Student> cmp =
    Comparator.comparingInt(
        (Student s) -> s.marks)
        .thenComparing(s -> s.name);
```

Comparable

```
class Student implements Comparable<Student> {
    @Override
    public int compareTo(Student o) {
        return Integer.compare(
            this.marks, o.marks);
    }
}
```

```
Now Arrays.sort(students); works directly.
```

Integer.compare()

```
Integer.compare(a, b);
```

Returns < 0, 0, or > 0.
Safer than a - b → avoids integer overflow.

Lambda Comparator

Instead of:

```
new Comparator<Student>() {
    public int compare(Student a, Student b){
        return Integer.compare(a.marks, b.marks);
    }
};
```

Write:

```
(a, b) -> Integer.compare(a.marks, b.marks);
```

Collections.reverse()

```
Collections.reverse(list);
```

Works on List<Integer>, **not** arrays.
Only **reverses** --- does **not** sort.

Reverse a Primitive Array

```
int i = 0, j = n - 1;
while (i < j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
    i++;
    j--;
}
```

```
Two-pointer swap --- works for every array type.
```

Quick Recall: sort → binarySearch → fill → copyOf/copyOfRange → equals/deepEquals → toString/deepToString → asList → clone